COMPUTATIONALLY EFFICIENT RECURRENT ARCHITECTURES FOR DISCRETIZED TRAJECTORY PREDICTION IN RESOURCE-CONSTRAINED SETTINGS

Daniil Losev (s246416), Antonio Riverso (s252132), Andrea Sanvito (s251931)

ABSTRACT

Trajectory prediction models are often based on large architectures and high-resolution spatial representations, making them unsuitable for embedded or resource-constrained systems. This work proposes an efficient, fully parametric sequence-to-one forecasting model designed to adapt to a wide range of computational budgets and operational needs. The model structure, window size, prediction horizon, hidden dimension, number of LSTM layers, grid resolution and even the number of Monte Carlo rollouts can all be tuned based on available resources. Despite operating with minimal configurations, such as a single LSTM layer with as few as 32 or 64 hidden units, the model maintains strong predictive performance. These results show that carefully designed lightweight architectures can provide reliable trajectory forecasting while remaining deployable in embedded or high-efficiency settings, such as drones or smart gadgets.

1. INTRODUCTION

1.1. Problem description

In recent years, progress in machine learning has been driven mainly by increasing model complexity: deeper architectures, wider layers, attention mechanisms and growing parameter counts. While these approaches improve accuracy in many domains, they come with very big computational complexity and energy requirements. Due to this, **efficiency** has often taken a back seat to raw performance. However, this trend is also trend contrast to the accelerating demand for models that operate directly on *embedded* or *low-power* devices, such as drones, wearables, smart sensors and autonomous edge platforms, where memory, energy and computational budgets are limited. In these settings, such complex architectures are for the most part impractical and impossible to deploy.

1.2. Goal

This work explores an alternative direction: instead of relying on large and sophisticated models, it is investigated how far it is possible to go with a minimalistic, highly efficient recurrent neural architecture designed specifically for constrained environments. The goal is not to push state-of-the-art accuracy through more performance, but to demonstrate that *careful problem formulation* and *structural simplifications* can produce models that are both **lightweight and effective**.

In this specific study the task of short-term trajectory prediction, a common problem in settings such as robotics, autonomous vehicles, UAV navigation and smart environmental sensing. In long-horizon forecasting complex dynamics and planning are required; on the other side, **short-term next-step prediction** is central to reactive control, collision avoidance and low-latency navigation. These are the situations in which embedded inference is essential: computing the next move must be fast, cheap and reliable.

2. METHODOLOGY

2.1. Data Wrangling

In this project, the *AIS dataset* [1] is used. It provides millions of daily samples containing attributes such as timestamp, geographic coordinates, ship type, speed, and heading angle. Due to the large size of the dataset, only one week of data (from July 1st to July 7th, 2025) is considered. Since AIS data is known to be noisy, all samples are processed through a dedicated *preprocessing* pipeline.

The first stage is data **sanitization**: samples with missing values, invalid formatting or MMSIs not following standard conventions are removed. The dataset is then restricted to a square region around Denmark, bounded by longitudes 0 to 20 and latitudes 52 to 60.

Additional **filtering** is employed to make sure only meaningful trajectories are kept: tracks with too few samples, insufficient displacement or very short duration are dropped.

The remaining, valid tracks are then divided into *segments* based on the time distance between samples. Each segment is linearly interpolated to obtain smooth trajectories and then resampled so that every new point occupies a grid cell adjacent to the previous one. To preserve temporal information, an additional attribute `dt` is added to each sample to represent the elapsed time relative to the previous.

2.2. Discretization

A key objective of this project is *efficiency*. With that in mind, we move from the *regression* task, predicting raw geographic

coordinates, to a *classification* one, where the space is transformed into a **grid** with a tunable resolution that can be adjusted as needed. In this study, a grid with cell dimensions of 0.01×0.01 degrees in latitude and longitude is adopted, corresponding to approximately one square kilometre per cell.

Therefore, the data produced by the preprocessing pipeline described in Section 2.1 are discretized into grid cell indices before being used for model training and inference.

2.3. Training Preprocessing

To enable models to effectively learn underlying patterns, a final andstructured preprocessing pipeline is applied to the data.

First, a small amount of **feature engineering** is applied: the course over ground (*COG*) attribute is transformed into its sine and cosine components, to ensure continuity between $0°$ and $360°$, and two additional attributes, $dSOG$ and $dCOG$, are introduced to represent the variations, with respect to the previous sample, of speed over ground and course over ground.

Subsequently, the dataset is first divided into *training* and *validation* splits, and then **normalized** using statistics computed exclusively on the training set, preventing *data leakage*. Each attribute is normalized according to its nature: *x*, *y*, *SOG* and *dSOG* are linearly scaled, mapping them to the interval $[0, 1]$; *dt* is normalized logarithmically to account for long stationary periods; *dCOG* is scaled by dividing by π, resulting in values within $[-1, 1]$. Sine and cosine components of *COG* already fall in the $[-1, 1]$ range and therefore require no normalization.

2.4. Models

Every model receives as input a 10-dimensional vector, produced by the pre-processing pipeline described in Section 2.3. Based on this input, the models perform two prediction tasks: a **classification** task over a 3×3 grid, representing the possible movements (remain in the same cell or move west, north, east, south or along their diagonal combinations), and a **regression** task to estimate all the remaining continuous attributes.

2.4.1. Baseline

As a baseline, we employed a simple, non-parametric model that predicts future vessel motion by averaging the most recent timesteps of the input trajectory. Given the last N observed steps, it computes the mean of all kinematic features (e.g., speed, $\sin\theta$, $\cos\theta$) and uses this vector as the continuous regression prediction. For direction classification, the model extracts the averaged heading components and normalizes them to obtain a unit movement vector, which is then compared via cosine similarity against a predefined set of nine canonical directions arranged on the 3×3 grid. These similarities are scaled to produce the class logits. Despite its simplicity, this baseline provides an interpretable and robust lower bound by capturing short-term motion persistence through recent averaging alone.

2.4.2. Sequence to One LSTM

As a first actual model, a *Recurrent Neural Network* with **Long Short-Term Memory** (LSTM) cells is adopted, as LSTMs represent a strong baseline for sequential modeling tasks, and are also widely used in trajectory prediction thanks to their ability in capturing long-term temporal relationships. The architecture contains neither attention mechanisms nor gating-heavy multi-layer stacks.

The model begins with a shallow **input-processing block** composed of a *linear layer*, a *ReLU activation*, and a *LayerNorm* operation. This stage projects heterogeneous features (numeric attributes and ship-type embeddings) into a shared representation space, stabilizing training results and simplifying the input distribution before it is fed to the LSTM layer. The ship type is encoded through a compact *embedding* layer, allowing the network to learn latent behavioural similarities between vessels.

The **core** of the network consists of one or more stacked *LSTM layers*. This component is responsible for modeling temporal patterns in the trajectory, but is, at the same time, the most computationally demanding part of the architecture.

Finally, the network outputs are produced through several small, specialized **prediction heads**, consisting in one *linear layer* per target group. A dedicated head predicts the discrete movement on the grid (*classification*), whereas additional heads handle the continuous *regression* targets. It is also important to notice that each head applies a different activation function, appropriate for the physical nature of its output (e.g., sigmoid for normalized speeds, tanh for trigonometric functions, softplus for non-negative time intervals). An example of this architecture is reported in Figure 1.

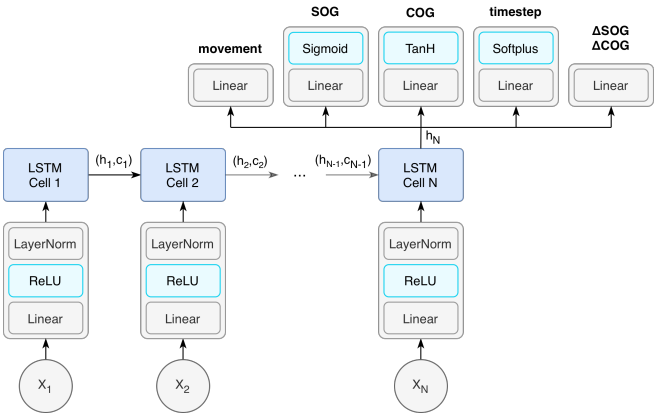

Fig. 1. Architecture of the sequence-to-one model.

2.4.3. Bidirectional, Sequence to Sequence, LSTM

As a reference for the performance limits of the sequence-to-one model, we constructed an expanded **Sequence-to-Sequence** architecture capable of predicting horizons equal in length to the input history.

The architecture utilizes a **bidirectional** LSTM encoder, a design specifically chosen to mitigate dependency on recent inputs and foster a global understanding of the entire trajectory, compressing it into a robust latent vector, which is subsequently passed to a decoder responsible for autoregressively generating the future trajectory. To improve training stability and limit error accumulation, *teacher forcing* is employed, probabilistically feeding ground-truth inputs to the decoder instead of its own past predictions. The teacher forcing ratio is linearly decayed over training (*curriculum learning*), gradually shifting from guided to fully autoregressive generation and reducing susceptibility to hallucinations.

Furthermore, to enhance feature processing, the same *input-processing block* described in Section 2.4.2 is employed in both the encoder and the decoder.

3. RESULTS

3.1. Evaluation

To find the best trade-off between performance and efficiency, several configurations of the Sequence-to-One model are tested, varying the *hidden dimension* and the *number of hidden layers*. The baseline and Sequence-to-Sequence architectures (see Section 2.4) are used respectively as a lower and an upper performance bound.

Models are *first* compared consdering the **accuracy** of the **greedy next-step prediction**, with results reported in Table 1.

As expected, the baseline model only achieves an accuracy of 37%, thus providing a lower limit and highlighting the importance of learning actual data patterns. Regarding the Sequence-to-One model, even the smallest tested configuration (32 hidden units, 1 layer) exceeds 80% accuracy across trajectories. Increasing the hidden dimension to 64 or 128 produces a higher accuracy, almost hitting the 83% threshold; increasing the number of LSTM layers does not produce relevant improvements and often leads to overfitting, as demonstrated by a lower validation accuracy. A higher window size was also tested, raising accuracy to nearly 85%: such a long input sequence is however computationally inefficient and impractical in real scenarios. The *Sequence-to-Sequence* model achieves a slightly higher accuracy, as expected from a more expressive and complex architecture. The small, 1% difference between the two models confirms that the simpler Sequence-to-One model is more than valid for trajectory prediction.

Model	Dimension	Layers	Window	Accuracy
baseline	-	-	20	37.00%
seq2one	32	1	20	80.82%
seq2one	32	4	20	80.42%
seq2one	64	1	20	81.60%
seq2one	64	2	20	81.65%
seq2one	128	1	20	82.99%
seq2one	128	2	20	82.73%
seq2one	256	1	20	82.24%
seq2one	128	1	100	84.88%
seq2seq	128	1	20	84.07%

Table 1. Accuracy metrics across different settings.

	Baseline		Seq2One		Seq2Seq	
Step	**Acc**	**Mean**	**Acc**	**Mean**	**Acc**	**Mean**
1	37.15	0.70	73.30	0.35	78.27	0.28
5	36.63	2.94	62.18	1.18	63.44	1.14
10	35.74	5.83	56.43	2.14	57.95	2.11
20	34.25	11.84	52.51	4.55	53.84	4.11
40	31.64	25.26	48.82	11.49	48.23	9.21

Table 2. Multi-step accuracies and mean distances from truth, for the different models.

Models are then compared with metrics that focus on the quality of **long-term predictions**. To this end, **accuracy** over a window size of 40 steps (or *two hours*, on average) is computed, together with the **mean distance** (in cells) from the ground truth, and the results are shown in Table 2. At step 20, the baseline model predicts only one third of all movements correctly, whereas the two models exceed one half. Moreover, where the baseline exhibits an average error above 25 cells, the trained models remain at 4.55 and 4.11 cells. In practice, this implies that a 20-steps-ahead prediction (approximately 20 km) locates the ship, on average, within a circle of radius just above 4 km. The same conclusions can be drawn when considering the 40 steps-ahead prediction: the accuracy of the baseline model underlines the necessity of trained models, and the Seq-to-One model confirms itself as a valid contender when compared with the highly complex Seq-to-Seq model.

3.2. Simulation

To evaluate the generative capabilities and robustness of the *Sequence-to-One LSTM* model, a **Monte Carlo simulation** framework is implemented. Unlike deterministic forecasting, this approach uses repeated stochastic rollouts to approximate the distribution of possible future trajectories. In the experiment, 75 independent trajectory simulations are generated using 20-step seeds taken from the validation set. Each rollout

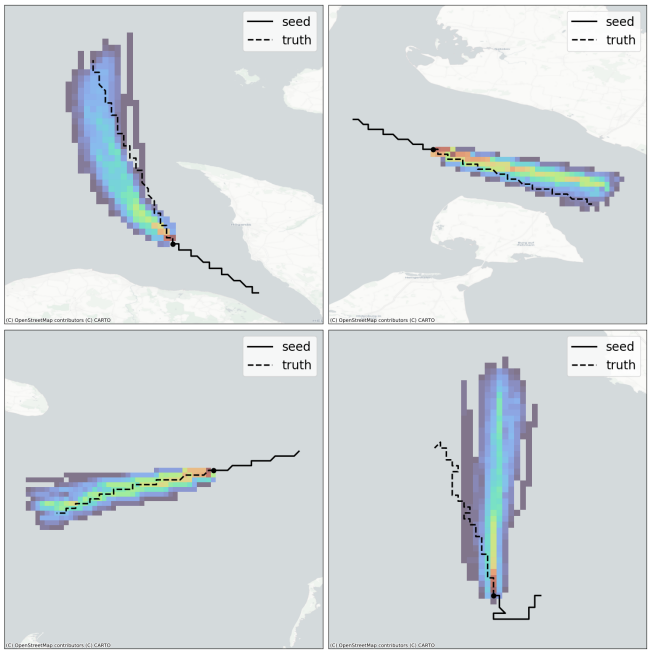

Fig. 2. Monte Carlo simulations for a sequence-to-one model with 1 LSTM-layer and 64 hidden units.

proceeds autoregressively: the model receives the previously predicted state, outputs a probability distribution over the nine grid movements, samples the next move and updates the vessel's physical attributes (speed, course, and *dt*) through the regression heads. The prediction horizon is set to 40 steps, corresponding to approximately **two hours of travel**.

The resulting rollouts, presented in Figure 2, highlight several characteristic behaviors of the model. In the Top-Left example, even though the seed suggests a straight path while the real route includes a sharp turn, the model still manages to follow the turn instead of continuing straight, showing that it *has learned* how vessels typically move in that area. In the Top-Right case, the predictions drift slightly toward the center of the channel, suggesting that the model has picked up the idea of keeping some distance from the coastline. The Bottom-Left example shows that the model maintains a *high accuracy* when the trajectory is regular and stable. Lastly, the Bottom-Right example shows both limits and robustness: despite starting from basically stationary data, the model eventually predicts a reasonable northward movement, even if the path is not perfectly matched to the true one.

4. DISCUSSION

4.1. Considerations

The results presented in Section 3 are very encouraging. The study provides a lightweight model made of only three small supporting layers and a single computationally heavier LSTM layer. The model, even if simple, remains highly **flexible** and can be adapted to different requirements.

Indeed, the grid representation is entirely **tunable**. The approach can obviously be extended to any geographical region and the grid resolution can be fully adjusted: smaller cells provide more precise predictions but increase computational cost, as more steps are required to generate a trajectory. Moreover, the model is **fully parametric** with respect to its internal architecture: even a configuration with 32 hidden units and a single layer proved sufficient to achieve strong performance. Training used a window size of just 20 steps, completed in around 10 minutes, and yielded a model that occupies less than 50 kB on disk, small enough to be deployed even on a micro-controller. Lastly, a relatively large number of rollouts were employed to generate Figure 2, but real scenarios can operate with far fewer simulations (even 1, with a greedy approach) without affecting performance, making the approach efficient even under tight computational constraints.

4.2. Future work

The choice to prioritize model simplicity and efficiency naturally comes with certain **limitations**.

One issue concerns physical **land boundaries**. Since the model was not trained with land–sea information, it can occasionally generate trajectories that cross onto land. To prevent this, a practical and efficient solution is to include a *sparse bitmap* distinguishing sea from land, actually enabling the model to avoid invalid regions. Another challenge emerges near **ports**, where ship behavior becomes more irregular due to arrivals, stays and departures. Because the model has no explicit notion of port locations, a lightweight improvement would be to add a boolean feature indicating whether a sample lies within a port area.

Finally, the model could also support **anomaly detection**, an aspect not explored in this work. Given its probabilistic output, the likelihood of the true trajectory could be computed at each step; if this value falls below a chosen *threshold*, also a tunable and setting-dependent parameter, the behavior could be flagged as anomalous.

5. CONCLUSION

In a world where models are becoming increasingly complex [2], this work demonstrates that strong performance can still be achieved with simple and lightweight architectures, allowing deployment of machine learning models on a wide range of environments. The study highlights the value of reducing complexity when it does not provide significant benefits and encourages a shift toward more efficient model-design choices. Overall, it offers a concrete and experimentally validated example of how deliberate simplification can support the development of high-efficiency and embedded machine learning systems.

6. REFERENCES

[1] Danish Maritime Authority, "AIS Data from Danish Waters", 2025. [Online]. Available: http://aisdata.ais.dk/.

[2] G. Menghani, "Efficient Deep Learning: A Survey on Making Deep Learning Models Smaller, Faster, and Better", 2023. Available: https://arxiv.org/abs/2106.08962.

[3] Code repository, hosted on GitHub. Available here.